

GOOGLE CTF 2024 | CATEGORIA REVERSING / CRYPTO

Google CTF Desafio Rusty School

Engenharia Reversa em Rust, Cifra Feistel de 4 Ramos e Cripto análise Algébrica em $GF(2^{16})$



INTEGRANTES DO GRUPO

José RLSF

Gabriel KN

GOOGLE CTF 2024 | CATEGORIA REVERSING / CRYPTO

Anatomia do Desafio: Rusty School

Engenharia Reversa em Rust, Cifra Feistel de 4 Ramos e Cripto análise Algébrica em $GF(2^{16})$

INTEGRANTES

José RLSF

Gabriel KN

Análise Técnica e Implementação



MÉTRICAS DO DESAFIO

Pontuação: 406 Pontos

Solves Globais: 6 Equipes

Arquivo Cifrado: 7.980 bytes (133 blocos)



ENTREGÁVEIS DO PROJETO

Solver: SageMath 10+ (multiprocessado)

Documentação: WriteUp, README, Slides
PDF

Flag: Reconstruída em Arte ASCII

Este projeto documenta a resolução completa do desafio Rusty School. A abordagem combina depuração dinâmica e patching do binário Rust no GDB (substituindo `/dev/urandom` por `/dev/zero`), modelagem de uma Cifra Feistel de 4 ramos e resolução de sistemas de equações quadráticas não-lineares em $GF(2^{16})$ no SageMath.

Contexto Competitivo & Visão Geral do Desafio

406

PONTUAÇÃO CTF

6

SOLVES GLOBAIS

133

BLOCOS CIFRADOS

~4 MIN

TEMPO DE EXECUÇÃO

OBJETIVO DO DESAFIO

- **Análise de Binário Proprietário:** Investigar o executável ELF x86-64 compilado em Rust (rustyschool).
- **Reversão de Cifra de Bloco:** A ferramenta processa dados em blocos de 48 bytes e adiciona uma semente de 12 bytes.
- **Decifração de Arquivo Grande:** Processar os 7.980 bytes cifrados contidos no arquivo flag.txt.encrypted.
- **Recuperação da Flag:** Reconstruir a mensagem original em formato de Arte ASCII de grande escala.

ARTEFATOS OFICIAIS DE ENTRADA

- rustyschool: Binário compilado em Rust (rustc 1.70.0) sem símbolos de depuração simplificados.
- flag.txt.encrypted: Arquivo cifrado contendo 133 blocos sequenciais de 60 bytes cada.
- solve.sage / solve.py: Solver algébrico baseado em variedades e Ideais Polinomiais no SageMath.
- solution/flag.txt: Artefato de saída com a bandeira final em Arte ASCII.

Diretriz Metodológica: Reversing First vs Crypto Pura

Diretriz:

'O trabalho de reversing precisa preceder a álgebra para que a apresentação não repita apenas Crypto.'

A relevância deste desafio está na ponte entre a engenharia reversa de baixo nível em Rust e a modelagem algébrica avançada.

ENGENHARIA REVERSA EM RUST (COMPLEXIDADE)

- **Boilerplate e Abstrações do Compilador:** O rustc insere rotinas extensas de verificação de limites, alocação de memória e tratamento de pânico.
- **Operador '?' e Result<T,E>:** Indireções de controle ruidosas que dificultam a leitura do pseudo-C gerado por decompiladores (IDA Pro / Ghidra).
- **Análise Dinâmica Obrigatória:** A inspeção estática é insuficiente; exige depuração direta no GDB e patching de chamadas de sistema.

CRIPTOANÁLISE ALGÉBRICA (DESAFIO MATEMÁTICO)

- **Cifra Feistel de 4 Ramos:** Fragmentação do bloco de 48 bytes em 4 ramos combinados com funções de hash MD5 e SHA-1.
- **Aritmética Modular de 96 Bits:** Operações controladas pelo primo constante $N = 2^{96} - 17$ (79160129948973046149879599747).
- **Sistemas em $GF(2^{16})$:** Evolução da chave derivada através de Ideais Polinomiais e variedades algébricas no SageMath.

Análise Estática & Estrutura do Binário Rust

PROPRIEDADES DA COMPILAÇÃO RUST

- **Arquivo ELF x86-64:** Não-stripado, mantendo nomes de funções auxiliares da biblioteca padrão do Rust.
- **Processamento por Blocos:** O binário lê o arquivo de entrada em segmentos fixos de **48 bytes** por iteração.
- **Estrutura do Output (60 bytes):** Cada bloco cifrado é composto por 48 bytes de ciphertext + **12 bytes de semente (rv1)** anexados ao final.
- **Dispersão Criptográfica:** Uso de hashes MD5, SHA-1 e exponenciação modular para difusão entre os ramos.

FLUXO DE CIFRAGEM DO BINÁRIO

```
// Pseudo-código do fluxo principal em Rust
fn encrypt_file(input: &Path;, output: &Path;) -> Result<()> {
    let mut chunk = [0u8; 48];
    while input.read_exact(&mut; chunk).is_ok() {
        let (cipher_48b, rv1_12b) = encrypt_chunk(&chunk;);
        output.write_all(&cipher;_48b)?;
        output.write_all(&rv1;_12b)?; // Append semente
    }
    Ok(())
}
```

Depuração Dinâmica & Controlando o Non-Determinismo

CONTROLE DE ALEATORIEDADE (/dev/urandom)

- **O Problema do Non-Determinismo:** A cada execução, o binário realizava chamadas a `getrandom / /dev/urandom` para gerar sementes aleatórias `K_init`.
- **A Solução por Patching:** Redirecionamento do ponteiro da string `/dev/urandom` para `/dev/zero` no binário ou via script GDB.
- **Resultado Obtido:** Permitiu obter sementes determinísticas zeradas durante a análise de reversão dos ramos.

ISOLAMENTO DE 1 RODADA NO GDB

```
# Script de depuração e patching no GDB
gdb ./rustyschool
(gdb) break *encrypt_chunk+0x120
(gdb) commands
    # Força o contador de rodadas Feistel para 1
    set $rax = 1
    continue
(gdb) end
(gdb) run input.txt output.enc
```

Arquitetura da Cifra Feistel de 4 Ramos (12 Rodadas)

Fracionamento do Bloco: Os 48 bytes de dados em cada iteração são fracionados em 4 ramos de 12 bytes (b_0 , b_1 , b_2 , b_3). A cada rodada, o estado é atualizado através de combinações não-lineares com hashes MD5 e SHA-1 e exponenciação modular de 96 bits.

RAMO 0 (MD5)

$$cb1 = b0 \oplus \text{MD5}(rv0 \parallel b2)[:12]$$

RAMO 1 (SHA-1)

$$cb2 = b1 \oplus \text{SHA1}(rv1 \parallel b2)[:12]$$

RAMO 2 (PERMUTAÇÃO)

b_2 propagado como entrada dos hashes.

RAMO 3 (MODULAR)

$$bs = ((b1 + b3) \cdot b0 + b2^{b1}) \bmod N$$

Inversão Algébrica do Ramo Modular (b3)

PROPRIEDADES DO PRIMO DE 96 BITS

- **Constante N de 96 bits:** 0xFFFFFFFFFFFFFFFFFFFFFFEF (Decimal: 79160129948973046149879599747).
- **Tamanho dos Operandos:** Os ramos b0, b1, b2, b3 possuem 12 bytes (96 bits) interpretados como inteiros little-endian.
- **Exponenciação Modular:** O termo $b2^{b1} \bmod N$ é calculado com exponenciação modular rápida.

FÓRMULA DE INVERSÃO DIRETA DE b3

Isolando algebricamente o ramo b3 no corpo primo $\mathbb{Z}_N$:

$$b3 = [((bs - b2^{b1} \bmod N) \cdot b0^{-1} \bmod N) - b1] \bmod N$$

- Esta inversão direta permite recuperar o ramo b3 de forma exata e determinística em cada rodada, sem necessidade de busca exaustiva!

Função de Derivação de Chaves `derive` em GF(2^{16})

CORPO FINITO & POLINÔMIO REDUTOR

- **Corpo de Galois GF(2^{16}):** Construído sobre o polinômio irreducible $x^{16} + x^5 + x^3 + x + 1$ (0x2B).
- **Vetor de Semente:** A semente de 12 bytes é dividida em 6 subchaves de 16 bits (a0, a1, a2, a3, a4, a5).
- **Sistema Quadrático:** Formulação de 6 equações quadráticas não-lineares por rodada para reconstruir a chave ancestral.

RESOLUÇÃO DE VARIEDADES EM SAGE

```
# Sistema de equações quadráticas em SageMath
G = PolynomialRing(F, names=('a0','a1','a2','a3','a4','a5')) a0, a1, a2, a3,
a4, a5 = G.gens()
eqs = [
    a0*a1 + a4 - a1*a2*a2*a3*a5 - c1,
    + a0 - c2, a3*a5 + a2*a1 - c3,
    - c4, a5*a0 + a3 - c5
]
my_id = Ideal(eqs) solutions =
my_id.variety()
```

Fase 0: Prova de Conceito e Validação do Bloco #0

Diretriz: *'Provar a inversão em um único bloco antes de recuperar o arquivo inteiro.'*

Esta etapa garante a validação da matemática de inversão dos 4 ramos e da chave em $GF(2^{16})$ antes de disparar o solver completo.

EXECUÇÃO DA PROVA DE CONCEITO

- **Isolamento dos 60 bytes Iniciais:** Leitura exclusiva do Bloco #0 do arquivo cifrado.
- **Extração da Semente rv1:** Leitura dos últimos 12 bytes do bloco.
- **Traversia Backtracking (DFS):** Inversão das 12 rodadas Feistel de trás para frente.
- **Validação de Saída:** Confirmação da recuperação de 48 bytes legíveis de texto plano.

LOG DE SAÍDA DO TERMINAL (FASE 0)

```
=====
[+] PROVA DE CONCEITO: Validando Bloco #0...
=====
[+] Sucesso! Bloco #0 decifrado (48 bytes):
    Texto:
    .d8888b. 888888888888 88888888888 .d888 88888888
=====
[+] Status: Validação com Sucesso!
```

Ajustes de Compatibilidade & Refatoração no SageMath 10+

DEPRECIAÇÕES NA VERSÃO SAGEMATH 10+

- **Remoção do Método `fetch_int`:** O método legado de conversão de inteiros para elementos de $\text{GF}(2^{16})$ gera exceções fatais na versão 10.9+.
- **Remoção de `integer_representation`:** A conversão inversa para inteiro foi substituída pela API moderna.
- **Independência do Pré-Processador:** Eliminação das macros `R.<x>` para permitir execução via Python 3 padrão.

TABELA DE REFATORAÇÃO DE CÓDIGO

- **Invocação Antiga (SageMath 9.x):**

```
F.fetch_int(val)
element.integer_representation()
```

- **Invocação Moderna (SageMath 10+):**

```
F.from_integer(int(val))
element.to_integer()
```

- **Anel de Polinômios Purificado:**

```
PolynomialRing(GF(2), names=('x',))
```

Execução Multiprocessada dos 133 Blocos & Flag Final

ORQUESTRADOR MULTIPROCESSADO (solve_fast.py)

- **Fatiamento em 133 Blocos:** O arquivo cifrado de 7.980 bytes é dividido em blocos individuais de 60 bytes.
- **Paralelismo com 12 Trabalhadores:** Uso do ProcessPoolExecutor para acelerar o cálculo das variedades algébricas.
- **Tempo de Conclusão:** Processamento de 100% do arquivo em apenas **~4 minutos**.
- **Montagem da Flag:** Reconstrução sequencial salva em solution/flag.txt.

BANNER DE SAÍDA (solution/flag.txt)

```
.d8888b. 888888888888 888888888888 .d888 88888888
d88P  Y88b 888 888 d88P" 888
888 888 888 888 d88P 888
888 888 888 8888888 d88P 888
888 88888 888 888 d88P 888
888 888 888 888 d88P 888
Y88b d88P 888 888 d88P 888
"Y8888P8" 888 888 d88P 8888888888
```

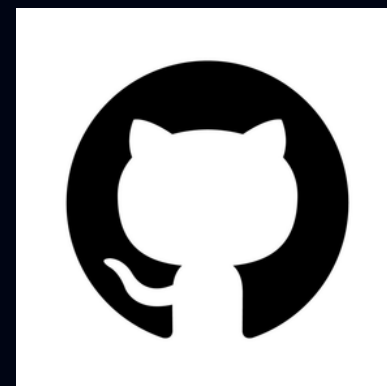
Estrutura do Repositório e Artefatos de Entrega

DIRETRIZES DO ROTEIRO

- **Limpeza e Organização do Repositório:** Manter apenas as pastas e arquivos estipulados.
- **Manifesto de Desafio (metadata.yaml):** Padronização seguindo o formato das competições de CTF.
- **Portabilidade da Solução:** Scripts testados para execução direta via python3 / sage.

ÁRVORE DE DIRETÓRIOS FINAL

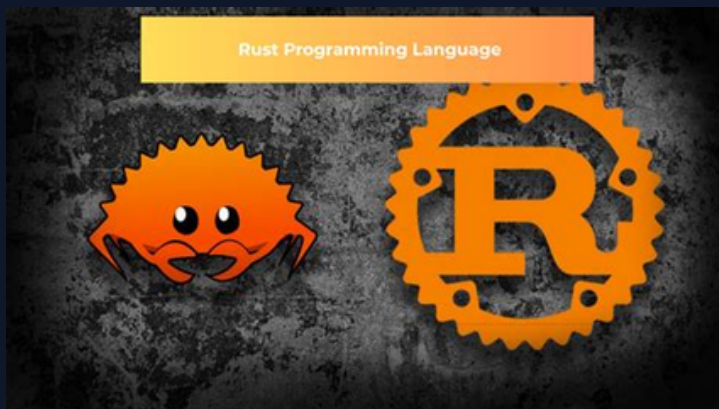
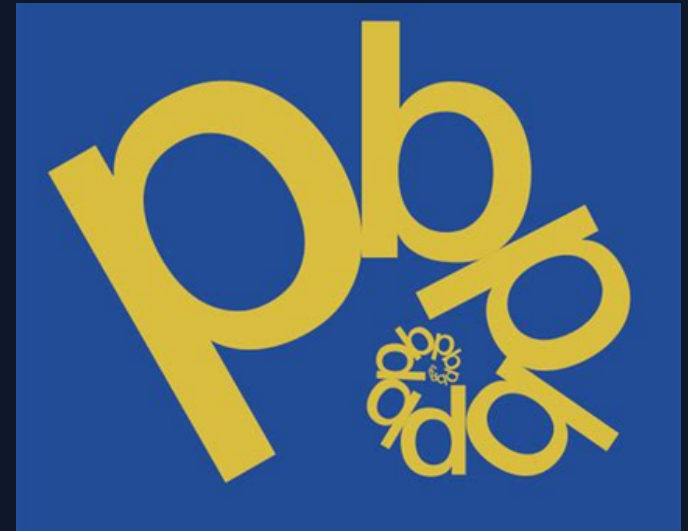
```
meu-repositorio-ctf/  
  ├── attachments/  
  │   ├── flag.txt.encrypted  
  │   └── rustyschool  
  ├── challenge/src/  
  ├── solution/  
  │   ├── flag.txt  
  │   ├── solve.sage  
  ├── metadata.yaml  
  ├── solve.py  
  └── solve_fast.py
```



Conclusão & Referências Técnicas

CONCLUSÕES TÉCNICAS

- **Sinergia Reversing + Crypto:** A análise do funcionamento foi essencial para entender o formato dos blocos antes de formular a matemática.
- **Validação Progressiva:** A Fase 0 (Bloco #0) evitou desperdício de tempo e confirmou o correto alinhamento das saídas.
- **Estabilidade da API:** A atualização dos métodos para SageMath 10+ assegurou a reprodutibilidade do projeto.



FONTES & REFERÊNCIAS OFICIAIS

- **Google CTF 2024 Repository:** [google-ctf/2024/quals/rev-rustyschool](https://github.com/google-ctf/2024/quals/rev-rustyschool)
- **Write-Up Oficial:** perfect blue CTF Team (superfashi, Riatre, cts)
- **Documentação SageMath:** Galois Fields, Multivariate Polynomial Rings & Ideals em $\text{GF}(2^{16})$